

Architectural Blueprint and Software Implementation for an Autonomous 6WD Swerve-Drive Wildfire Rover

1. Introduction and Operational Paradigm

The deployment of autonomous ground vehicles in high-risk, unstructured environments—specifically for wildland fire risk assessment and real-time propagation modeling—requires a resilient hardware architecture tightly coupled with a deterministic, highly optimized software stack. The objective of this comprehensive report is to define the exact engineering specifications, mathematical models, and production-grade Python implementations for a 6-Wheel Drive (6WD) Swerve-Drive Rover. This platform operates on ZorinOS (a Debian-based Linux distribution) executing on a Raspberry Pi 4 microprocessor. Wildfire propagation is governed by complex, non-linear interactions between topography, meteorological conditions, and fuel aridity. Traditional orbital and aerial remote sensing platforms suffer from temporal latency and occlusion by canopy cover or smoke. An autonomous ground rover bridges this data gap by capturing micro-climate telemetry at the surface level. By integrating highly constrained actuator multiplexing, multi-modal sensor fusion, real-time computer vision, stochastic cellular automata simulation, and Edge AI inference, the rover acts as a mobile computational node. The resulting telemetry is serialized into highly compact packets for long-range, low-power LoRa transmission, providing critical situational awareness to incident commanders.

This document serves as the foundational architectural blueprint, detailing the hardware limitations, the inverse kinematics geometry, the sensor fusion mathematics, the physical modeling of fire spread, and the asynchronous software orchestration required to operate the system safely and efficiently.

2. Hardware Architecture, Integration, and Constraints

The engineering of an autonomous rover on a constrained embedded platform like the Raspberry Pi 4 necessitates strict resource management. While the Pi 4 offers a quad-core ARM Cortex-A72 processor, its General Purpose Input/Output (GPIO) pinout and hardware pulse-width modulation (PWM) channels are limited. Therefore, an extensive network of Inter-Integrated Circuit (I2C) expansions and logic multiplexing must be employed.

2.1 Microprocessor and Operating System Ecosystem

The computational core is the Raspberry Pi 4 Model B. ZorinOS, being derived from Debian/Ubuntu, provides a robust, predictable kernel environment that ensures deep compatibility with hardware-interfacing libraries such as `smbus2`, `RPi.GPIO`, and the

Adafruit-Blinka hardware abstraction layer. Given the computational density of concurrent tasks—ranging from high-frequency 2kHz PWM generation for motor torque to matrix-heavy Computer Vision (CV) for fuel assessment—the system relies heavily on Python's asyncio library. Asynchronous programming prevents input/output (I/O) blocking across isolated hardware threads, ensuring that a delay in reading an I2C sensor does not interrupt the locomotion control loop.

2.2 Propulsion Topologies and Actuator Optimization

The 6WD platform utilizes six GA25-370 DC gear motors, chosen for their high torque output and durability in rugged terrain. These are driven by three L298N dual H-bridge motor drivers. Each L298N driver is capable of controlling two DC motors independently. Conventionally, controlling three L298N drivers requires twelve logic signals (IN1, IN2, IN3, IN4 for each of the three boards) to dictate motor direction, alongside six PWM signals (ENA, ENB) to govern torque and velocity.¹

To conserve the limited GPIO pins on the Raspberry Pi 4, a specific hardware multiplexing strategy is employed: the logic pins for all three L298N drivers are physically tied together in parallel on the printed circuit board (PCB) routing.

GPIO Mapping for Tied Logic Pins

By unifying the logic states, all six wheels share the same rotational direction polarity relative to the chassis at the electrical level. The mapping is consolidated as follows:

L298N Logic Pin	Raspberry Pi GPIO (BCM)	Locomotion Function	Shared Across Hardware
IN1	GPIO 5	Forward Logic (Side A - Left Wheels)	Driver 1, 2, 3
IN2	GPIO 6	Reverse Logic (Side A - Left Wheels)	Driver 1, 2, 3
IN3	GPIO 13	Forward Logic (Side B - Right Wheels)	Driver 1, 2, 3
IN4	GPIO 19	Reverse Logic (Side B - Right Wheels)	Driver 1, 2, 3

The Kinematic Workaround for Tied Logic: Because all wheels on a given side are electrically forced into the same drive direction state, achieving complex rotational maneuvers or reverse translational vectors for individual wheels requires a mechanical workaround. If the kinematic engine mathematically requests a negative velocity for a specific wheel (e.g., during a zero-radius turn or lateral strafing), the electrical velocity magnitude remains strictly positive.

Instead, the associated steering servo is rotated exactly 180° (π radians) to invert the physical thrust vector.³ This transfers the complexity of differential movement from the limited GPIO logic pins to the highly capable I2C servo controller.

High-Frequency Speed Control via I2C (Board B)

The enable pins (ENA and ENB) across the three L298N drivers are routed to a dedicated PCA9685 16-channel 12-bit PWM controller (designated as Board B). The I2C base address for the PCA9685 is 0x40. To prevent address collision, Board B is configured to address 0x41 by creating a solder bridge across the A0 hardware pad.⁴

Board B is explicitly initialized to operate at a frequency of 2kHz. Driving the inductive coils of the GA25-370 DC motors at 2kHz maximizes torque efficiency, minimizes audible acoustic switching noise (coil whine), and provides high-resolution speed control without taxing the Pi's CPU with software PWM.⁶

2.3 Steering Actuation Topologies

Omnidirectional swerve steering is achieved using six MG90S metal-gear micro servos,

capable of continuous 360° vertical rotation. These servos are connected to the primary PCA9685 controller (designated as Board A) operating at the default 0x40 address.⁵ Board A is daisy-chained via the I2C bus to Board B.

Unlike DC motors, standard servos expect a control pulse every 20 milliseconds, necessitating a strict 50Hz PWM frequency. Mixing 50Hz servos and 2kHz DC motor drivers on a single PCA9685 is mathematically impossible due to the shared internal oscillator pre-scaler; thus, the dual-board daisy-chain topology is a strict hardware requirement.⁶

2.4 Multi-Modal Sensor Array Integration

The rover's sensory payload is designed to capture the necessary variables for the Rothermel-based cellular automata model: fuel state, topography, and wind vectors.

1. **Gas and VOC Detection:** An MQ135 analog gas sensor detects atmospheric volatile organic compounds (VOCs), carbon monoxide, and smoke particulates. Because the Raspberry Pi 4 lacks native hardware Analog-to-Digital Converters (ADCs), an ADS1115 16-bit I2C ADC is utilized to sample the MQ135's analog output voltage with high precision.⁷
2. **Anemometer and Wind Vane:** A custom electromechanical array of four SS41F digital bipolar Hall-effect sensors generates an 8-point compass vector representing wind direction.⁸ A fifth Hall sensor is utilized for RPM pulse-counting, firing a hardware interrupt on the Pi to calculate relative wind velocity.⁹
3. **Inertial and Topographical Navigation:** An MPU6050 6-Degrees-of-Freedom (6-DoF) IMU communicates via I2C to provide raw acceleration and angular velocity data. This is fused mathematically with a Neo 6M GPS module communicating via UART to establish spatial localization and topographical surface normals (pitch/slope).¹¹
4. **Computer Vision:** A standard USB camera captures RGB frames. These matrices are processed using OpenCV to calculate the Normalized Green-Red Difference Index (NGRDI), a reliable optical metric for assessing vegetation aridity and fuel load.¹³
5. **Data Telemetry:** An RF96 (SX1276) LoRa module operates over the Serial Peripheral

Interface (SPI). It provides low-bandwidth, high-penetration RF transmission, broadcasting semantic JSON packets summarizing the rover's findings to a centralized command node.¹⁴

3. Subsystem A: The Kinematics Engine

The kinematics engine acts as the translation layer between high-level path planning and low-level actuator commands. The inverse kinematics for a 6-wheel swerve drive translates desired chassis velocities—translational velocity in the x-axis (V_x), translational velocity in the y-axis (V_y), and rotational velocity (ω)—into specific speeds and angles for each of the six independent wheel modules.³

3.1 Mathematical Formulation and Overdetermination

Let (x_i, y_i) be the Cartesian coordinates of wheel i relative to the geometric center of the rover chassis. The required velocity vector components for wheel i are calculated using the following transformations:

$$v_{x,i} = V_x - \omega \cdot y_i$$

$$v_{y,i} = V_y + \omega \cdot x_i$$

From these individual vectors, the magnitude (wheel speed) v_i and the steering angle δ_i are derived using trigonometric resolution:

$$v_i = \sqrt{v_{x,i}^2 + v_{y,i}^2}$$

$$\delta_i = \text{atan2}(v_{y,i}, v_{x,i})$$

Because a 6WD platform has 12 degrees of freedom (6 steering actuators, 6 driving actuators)

but only requires 3 parameters (V_x, V_y, ω) to specify rigid body chassis motion, the mathematical model is heavily overdetermined.¹⁵ If the physical wheel modules are not aligned perfectly with the mathematical ideal output by these equations, mechanical slip, wheel scrubbing, and severe current spikes in the L298N drivers will occur.

Furthermore, because of the GPIO hardware constraint where all motor direction logic pins are tied together, v_i must be strictly maintained as an absolute positive value. If the vector math

requires a negative thrust relative to the wheel's current forward-facing orientation, δ_i is shifted by π radians (180°) to reverse the force vector mechanically. Finally, if the kinematic output requests a speed that exceeds the maximum physical capability of the motors (PWM saturation), all wheel speeds must be proportionally desaturated (normalized) to maintain the intended trajectory vector without distortion.¹⁶

3.2 Python Implementation (kinematics.py)

The following module implements the inverse kinematics with production-grade type hinting, geometric optimizations, and error handling.

Python

```
import math
import logging
from dataclasses import dataclass
from typing import List, Tuple

# Configure module-level logger for production debugging
logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

@dataclass
class SwerveModuleState:
    """Represents the target state for a single swerve module."""
    speed: float
    angle: float

class SixWheelSwerveKinematics:
    def __init__(self, track_width: float, wheel_base: float):
        """
        Initializes the geometry of the 6WD swerve chassis.
        Assumes a symmetrical rectangular layout with middle wheels centered.

        :param track_width: Physical distance between left and right wheels (meters).
        :param wheel_base: Physical distance between front and back wheels (meters).
        """
        if track_width <= 0 or wheel_base <= 0:
            logger.error("Invalid chassis dimensions provided.")
            raise ValueError("Dimensions must be positive non-zero floats.")
```

```
hw = track_width / 2.0
hl = wheel_base / 2.0
```

```
# Coordinates (x, y) for the 6 wheels relative to the chassis geometric center
# X is forward/backward, Y is left/right
self.wheel_positions =
logger.info(f"Initialized 6WD Kinematics Matrix: {self.wheel_positions}")
```

```
def calculate_inverse_kinematics(self, vx: float, vy: float, omega: float) -> List:
```

```
    """
```

```
    Converts a desired chassis velocity vector into individual module states.
```

```
:param vx: Target chassis velocity in X axis (m/s).
```

```
:param vy: Target chassis velocity in Y axis (m/s).
```

```
:param omega: Target chassis rotational velocity (rad/s).
```

```
:return: List of SwerveModuleState objects containing speed and angle.
```

```
    """
```

```
    states =
```

```
    max_speed = 0.0
```

```
    try:
```

```
        for x, y in self.wheel_positions:
```

```
            # Calculate independent wheel velocity vectors
```

```
            v_x_i = vx - omega * y
```

```
            v_y_i = vy + omega * x
```

```
            # Derive magnitude and angle
```

```
            speed = math.hypot(v_x_i, v_y_i)
```

```
            angle = math.atan2(v_y_i, v_x_i)
```

```
            # Hardware Optimization: Ensure speed is always positive.
```

```
            # Because the L298N logic pins are physically tied together,
```

```
            # all motors are locked to 'forward' electrical polarity.
```

```
            # If the calculated angle lies outside the front-facing arc (-90 to 90 deg),
```

```
            # we invert the steering servo by 180 degrees to achieve reverse mechanical thrust.
```

```
            if speed > 0.001: # Deadband threshold to prevent jitter
```

```
                # Normalize angle to range [-pi, pi]
```

```
                angle = (angle + math.pi) % (2 * math.pi) - math.pi
```

```
                if angle > (math.pi / 2) or angle < -(math.pi / 2):
```

```
                    # Shift angle by 180 degrees
```

```
                    angle = (angle + math.pi) % (2 * math.pi) - math.pi
```

```
                    # The magnitude (speed) remains positive.
```

```

    if speed > max_speed:
        max_speed = speed

    states.append(SwerveModuleState(speed=speed, angle=angle))

    # Desaturation: If any wheel exceeds the theoretical maximum PWM limit (1.0),
    # proportionally reduce all wheel speeds to preserve the chassis vector ratio.
    if max_speed > 1.0:
        logger.debug(f"Kinematic saturation detected (max {max_speed:.2f}). Desaturating
array.")
        for state in states:
            state.speed /= max_speed

    return states

except Exception as e:
    logger.critical(f"Math error in kinematics resolution: {e}")
    # Fail-safe: Return zeroed states to halt the rover
    return

```

4. Subsystem B: Multi-Modal Sensor Fusion and Geospatial Mapping

The mapping engine is responsible for contextualizing the rover's environment. It fuses raw electronic signals—susceptible to electrical noise, mechanical vibration, and signal bounce—into coherent, geospatially accurate data structures.

4.1 Inertial Measurement and the Complementary Filter

The MPU6050 MEMS sensor provides 3-axis linear acceleration and 3-axis angular velocity.¹⁸ Accelerometers accurately measure the earth's gravity vector to determine pitch and roll, but they are highly susceptible to high-frequency vibrational noise from the 6WD chassis traversing rough terrain. Conversely, gyroscopes provide highly responsive angular velocity tracking but suffer from low-frequency integration drift over time, losing absolute reference to the horizon.¹²

The Complementary Filter acts as a computationally inexpensive, deterministic Sensor Fusion algorithm. It applies a low-pass filter to the accelerometer data (rejecting vibration) and a high-pass filter to the gyroscope data (rejecting drift).¹⁹

The recursive formula for the surface normal (e.g., pitch) is defined as:

$$\theta = \alpha \cdot (\theta + \omega_{gyro} \cdot \Delta t) + (1 - \alpha) \cdot \theta_{acc}$$

Where α is the filter coefficient (typically 0.98), ω_{gyro} is the angular rate, and θ_{acc} is the absolute angle derived from the gravity vector. GPS velocity data from the Neo 6M module provides long-term linear translation vectors, which can be subtracted from the accelerometer data to eliminate phantom tilt caused by centripetal acceleration during cornering. The resulting accurate surface normal directly feeds into the stochastic simulation's slope modifier.²¹

4.2 SS41F Wind Vane Truth Table Logic

An array of four SS41F bipolar Hall-effect sensors (North, East, South, West) is arranged around a central axis.⁸ A rotating bipolar magnet connected to the physical wind vane sweeps across them. The SS41F operates by transitioning its open-collector output LOW when exposed to a

magnetic flux density exceeding its operate point (B_{op} , typically 40 to 150 Gauss).¹⁰

By carefully spacing the sensors relative to the magnet's physical width, an 8-point directional resolution is achieved. When the magnet aligns exactly between two sensors, the magnetic flux triggers both simultaneously.²³

Wind Direction Logic Truth Table (Active-Low):

North	East	South	West	Decoded Vector	Angle (°)
0	1	1	1	North	0.0
0	0	1	1	North-East	45.0
1	0	1	1	East	90.0
1	0	0	1	South-East	135.0
1	1	0	1	South	180.0
1	1	0	0	South-West	225.0
1	1	1	0	West	270.0
0	1	1	0	North-West	315.0

A fifth SS41F sensor acts as an anemometer. As a separate multi-pole ring magnet spins with the wind cups, the sensor toggles. The Raspberry Pi utilizes edge-detection hardware

interrupts to count these pulses over time (Δt), directly yielding RPM, which is scaled by a

radius constant to determine wind velocity (V_{wind}) in m/s.²⁴

4.3 NGRDI Vision Processing for Fuel Assessment

The Normalized Green-Red Difference Index (NGRDI) leverages standard RGB optical imaging to estimate vegetation health, biomass, and aridity.²⁵ Dry, highly combustible grass and dead

timber (Non-Photosynthetic Vegetation, NPV) reflect high volumes of red wavelengths due to the exposed cellulose and lignin. Conversely, healthy, hydrated green vegetation (GV) absorbs red light for photosynthesis and reflects green.²⁶

Using OpenCV, the matrix calculation is performed rapidly:

$$\text{NGRDI} = \frac{\text{Green} - \text{Red}}{\text{Green} + \text{Red}}$$

Values approaching -1.0 indicate extreme aridity (high fire risk), while values approaching 1.0 indicate lush, hydrated flora.

4.4 Geospatial Data Storage (Uber H3)

All synthesized sensor telemetry (Elevation, Aridity/NGRDI, Wind, Gas PPM) is mapped to absolute global coordinates using the GPS module and aggregated into Uber's H3 hierarchical hexagonal grid system.²⁷

Hexagons are fundamentally superior to standard square grids for spatial simulation. In a square grid, the distance from the center of a cell to its orthogonal neighbors is different from the distance to its diagonal neighbors, creating artificial distortions in wave propagation. In a hexagonal grid, the distance from the center to all six neighbors is identical.²⁸ This equidistant topological property is a strict requirement for mathematically sound, isotropic fire spread modeling via cellular automata.²⁹

4.5 Python Implementation (mapping.py)

Python

```
import time
import math
import logging
import numpy as np
import cv2
import h3
import board
import busio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn

logger = logging.getLogger(__name__)

class SensorFusionEngine:
```

```

def __init__(self, alpha: float = 0.98):
    self.alpha = alpha
    self.pitch = 0.0
    self.roll = 0.0
    self.last_time = time.time()

    # I2C Initialization for ADS1115 (MQ135 Gas Sensor)
    try:
        self.i2c = busio.I2C(board.SCL, board.SDA)
        self.ads = ADS.ADS1115(self.i2c)
        # MQ135 is connected to AIN0 on the ADS1115
        self.mq135_channel = AnalogIn(self.ads, ADS.P0)
        logger.info("ADS1115 ADC successfully initialized on I2C bus.")
    except Exception as e:
        logger.error(f"Failed to initialize ADC: {e}")
        self.ads = None

def update_complementary_filter(self, accel_data: tuple, gyro_data: tuple) -> tuple:
    """
    Fuses MPU6050 Accelerometer and Gyroscope data to extract stable surface normals.

    :param accel_data: Tuple of (ax, ay, az) in standard g-forces.
    :param gyro_data: Tuple of (gx, gy, gz) in degrees per second.
    :return: (pitch, roll) in degrees.
    """
    current_time = time.time()
    dt = current_time - self.last_time
    self.last_time = current_time

    ax, ay, az = accel_data
    gx, gy, gz = gyro_data

    try:
        # Derive absolute tilt angle from gravity vector
        accel_pitch = math.degrees(math.atan2(ay, math.sqrt(ax**2 + az**2)))
        accel_roll = math.degrees(math.atan2(-ax, math.sqrt(ay**2 + az**2)))

        # Apply Complementary Filter logic
        self.pitch = self.alpha * (self.pitch + gx * dt) + (1 - self.alpha) * accel_pitch
        self.roll = self.alpha * (self.roll + gy * dt) + (1 - self.alpha) * accel_roll

    return self.pitch, self.roll
except ZeroDivisionError:

```

```
logger.warning("Zero division encountered in IMU filter matrix. Returning previous state.")
return self.pitch, self.roll
```

```
def parse_wind_direction(self, n: int, e: int, s: int, w: int) -> float:
```

```
    """
```

```
    Parses 4x SS41F Hall-effect sensors into an absolute wind direction angle.
```

```
    Note: SS41F is Active-Low. 0 indicates magnetic field presence.
```

```
    """
```

```
    states = (n, e, s, w)
```

```
    direction_map = {
```

```
        (0, 1, 1, 1): 0.0, # N
```

```
        (0, 0, 1, 1): 45.0, # NE
```

```
        (1, 0, 1, 1): 90.0, # E
```

```
        (1, 0, 0, 1): 135.0, # SE
```

```
        (1, 1, 0, 1): 180.0, # S
```

```
        (1, 1, 0, 0): 225.0, # SW
```

```
        (1, 1, 1, 0): 270.0, # W
```

```
        (0, 1, 1, 0): 315.0 # NW
```

```
    }
```

```
    angle = direction_map.get(states, None)
```

```
    if angle is None:
```

```
        logger.debug(f"Ambiguous Hall sensor state detected: {states}")
```

```
    return angle
```

```
def calculate_ngrdi(self, frame: np.ndarray) -> float:
```

```
    """
```

```
    Calculates NGRDI scalar from an optical frame to detect dry vegetative fuel.
```

```
    """
```

```
    try:
```

```
        # OpenCV loads matrices in BGR format
```

```
        b, g, r = cv2.split(frame.astype(np.float32))
```

```
        # Add epsilon (1e-6) to denominator to prevent division by zero in numpy
```

```
        denominator = g + r + 1e-6
```

```
        ngrdi_matrix = (g - r) / denominator
```

```
        # Return spatial average of NGRDI across the entire field of view
```

```
        return float(np.mean(ngrdi_matrix))
```

```
    except Exception as e:
```

```
        logger.error(f"Image processing fault during NGRDI extraction: {e}")
```

```
        return 0.0
```

```

def map_to_hexgrid(self, lat: float, lon: float, attributes: dict, resolution: int = 10) -> dict:
    """
    Compresses spatial telemetry into Uber H3 Geospatial Matrix node.
    """
    try:
        # Generate deterministic Hexagon ID from absolute coordinates
        hex_id = h3.geo_to_h3(lat, lon, resolution)

        hex_data = {
            "hex_id": hex_id,
            "elevation_pitch": attributes.get("pitch", 0.0),
            "fuel_state_ngrdi": attributes.get("ngrdi", 0.0),
            "wind_dir": attributes.get("wind_dir", 0.0),
            "gas_ppm": attributes.get("gas_v", 0.0)
        }
        return hex_data
    except Exception as e:
        logger.error(f"Failed to generate H3 geometry: {e}")
        return {}

```

5. Subsystem C: Wildfire Propagation Simulation

Wildland fire spread is heavily reliant on stochastic thermodynamics. To computationally forecast geographic propagation based on the rover's localized sensor data, the physical phenomenon is mathematically abstracted using a two-dimensional Stochastic Cellular Automata (CA) model.³⁰

Unlike traditional continuous deterministic engines (such as the standard implementation of FARSITE or Prometheus) which treat fire as an expanding elliptical wavefront using Huygens' principle³¹, a stochastic CA evaluates the probability of state transitions on a discrete, per-cell basis. This probabilistic nature natively captures the erratic, unpredictable behavior of spotting and micro-climate turbulence inherent to wildland fires.³⁴

5.1 Cellular Automata State Architecture

Each hexagonal cell in the H3 geospatial grid assumes one of three discrete thermodynamic states during the simulation:

- S_0 : Unburned (Fresh, unignited fuel load)
- S_1 : Burning (Currently undergoing combustion and emitting thermal/radiative energy)
- S_2 : Ashes (Depleted fuel load, incapable of re-ignition)

At each discrete temporal step t , the transition of an adjacent cell from S_0 to S_1 is governed by the comprehensive burn probability p_{burn} . When a cell completes its temporal burn window, it deterministically transitions from S_1 to S_2 .

5.2 The Probability Function and Modifiers

Based on a hybridized derivation of the fundamental Rothermel empirical fire spread equations³⁰, the likelihood of propagation from a burning cell to a neighboring unburned cell is calculated as:

$$p_{burn} = p_0 \cdot (1 + p_{veg}) \cdot p_w \cdot p_s$$

Where p_0 represents the baseline ignition probability under flat, windless, and nominal fuel conditions. The term p_{veg} acts as the fuel modifier, which is mapped inversely from the rover's NGRDI vision index (as a highly negative NGRDI correlates to highly combustible NPV).

The Exponential Wind Modifier (p_w)

Wind introduces a long-range convective preheating flux, physically pushing the flame column toward unburned fuel and accelerating dehydration. Extensive wind-tunnel and empirical studies demonstrate that the rate of fire spread scales exponentially with wind vector alignment.³³ The modifier is calculated as:

$$p_w = e^{c \cdot V \cdot \cos(\theta_{wind} - \theta_{spread})}$$

Where c is an environmental calibration constant, V is wind velocity in m/s (derived from the Hall sensor RPM interrupts), θ_{wind} is the absolute wind vector, and θ_{spread} is the geometric heading angle between the S_1 source cell and the S_0 target cell. If the wind blows directly toward the target cell, the cosine evaluates to 1, maximizing the exponential amplification.

The Trigonometric Slope Modifier (p_s)

Topographical slope radically accelerates fire spread uphill. Uphill propagation physically aligns the unburned fuel with the convective heat column and drastically decreases the distance required for radiative heat transfer to cause auto-ignition.³³ The terrain modifier is:

$$p_s = 1 + \alpha \cdot \tan(\theta_{slope})$$

Where θ_{slope} is the surface normal angle (pitch) determined by the MPU6050 complementary filter. The constant α scales the trigonometric response. This ensures that a 30° positive slope results in massive geometric acceleration compared to flat terrain.

5.3 Python Implementation (simulation.py)

Python

```
import numpy as np
import math
import logging

logger = logging.getLogger(__name__)

class CellularAutomataWildfire:
    def __init__(self, grid_size: tuple):
        """
        Initializes the spatial matrix.
        States: 0 = Unburned, 1 = Burning, 2 = Ashes.
        """
        self.grid = np.zeros(grid_size, dtype=np.int8)
        self.rows, self.cols = grid_size

        # Thermodynamic constants derived from hybridized Rothermel framework
        self.p0 = 0.15      # Baseline ignition probability
        self.wind_c = 0.05  # Wind exponential scaling constant
        self.slope_alpha = 2.0 # Slope trigonometric amplification constant

        logger.info(f"Initialized Stochastic CA Grid of shape {grid_size}")

    def calculate_p_burn(self, p_veg: float, wind_v: float, wind_dir: float,
                        slope_angle: float, spread_angle: float) -> float:
        """
        Executes the stochastic probability calculus for a given Moore neighborhood interaction.
        """
        try:
            # Wind modifier (Exponential Convective Preheating)
            angle_diff = math.radians(wind_dir - spread_angle)
            p_w = math.exp(self.wind_c * wind_v * math.cos(angle_diff))
```

```

# Slope modifier (Trigonometric Radiative Preheating)
# Tangent evaluates to 0 on flat terrain, leaving the 1.0 base.
p_s = 1.0 + self.slope_alpha * math.tan(math.radians(slope_angle))

# Clamp negative slope suppression to avoid mathematically impossible probabilities
p_s = max(0.1, p_s)

# Aggregate Probability
p_burn = self.p0 * (1.0 + p_veg) * p_w * p_s
return min(p_burn, 1.0) # Absolute physical cap at 100% ignition guarantee

except Exception as e:
    logger.error(f"Math domain error in probability calculation: {e}")
    return 0.0

def step(self, environmental_data: dict):
    """
    Advances the CA state machine by one discrete temporal step.
    """
    new_grid = self.grid.copy()

    # Extract localized micro-climate telemetry
    wind_v = environmental_data.get('wind_speed', 0.0)
    wind_dir = environmental_data.get('wind_dir', 0.0)
    # Invert NGRDI mapping: healthy vegetation (positive) lowers ignition chance
    p_veg = 1.0 - environmental_data.get('ngrdi', 0.0)
    slope = environmental_data.get('pitch', 0.0)

    for r in range(self.rows):
        for c in range(self.cols):
            if self.grid[r, c] == 1:
                # Cell is burning. Evaluate probabilistic spread to 8-cell Moore neighborhood
                for dr in [-1, 0, 1]:
                    for dc in [-1, 0, 1]:
                        if dr == 0 and dc == 0:
                            continue # Skip self

                        nr, nc = r + dr, c + dc
                        if 0 <= nr < self.rows and 0 <= nc < self.cols:
                            if self.grid[nr, nc] == 0: # Target is unburned fuel
                                # Determine geometric heading of spread
                                spread_angle = math.degrees(math.atan2(dr, dc)) % 360

```

```

p_burn = self.calculate_p_burn(p_veg, wind_v, wind_dir, slope, spread_angle)

# Stochastic Monte-Carlo threshold evaluation
if np.random.random() <= p_burn:
    new_grid[nr, nc] = 1 # Ignite target

# Thermal depletion: Burning cell exhausts fuel and turns to ashes
new_grid[r, c] = 2

self.grid = new_grid

```

6. Subsystem D: Edge AI Inference, Telemetry, and Orchestration

The final execution environment consolidates locomotion, mapping, and physics simulation. The output of the cellular automata generates a multidimensional grid of predicted fire spread. This raw matrix is too dense for remote transmission and requires immediate edge-based classification to generate a categorical risk assessment ("High", "Medium", "Low") before serialization via LoRa.

6.1 Edge AI Integration (TensorFlow Lite)

While the NGRDI visual index serves as a primary deterministic heuristic, identifying the complex, non-linear relationships between fuel structure, evolving slope, and historical weather patterns requires deep learning. A pre-compiled, INT8-quantized .tflite model acts as the inference engine.⁴⁰

TensorFlow Lite is inherently optimized for the ARM Cortex-A72 architecture of the Raspberry Pi 4. By quantizing weights to 8-bit integers, the model maintains a trivial RAM footprint and executes inference in milliseconds, entirely avoiding thermal throttling.⁴² The model ingests the final 2D array output of the Cellular Automata simulation and maps it to the categorical danger classes.

6.2 LoRa Semantic Serialization and Chirp Spread Spectrum

The RF96 (SX1276) LoRa module utilizes Chirp Spread Spectrum (CSS) modulation. While LoRa exhibits extreme range and obstacle penetration capabilities—vital for traversing dense forest canopies—it is severely bandwidth-constrained.⁴³ LoRaWAN and raw LoRa packets possess a hard physical Maximum Transmission Unit (MTU) limit of 256 bytes per packet.⁴⁴ Standard, verbose JSON formatting easily surpasses this limit when appending overhead such as MAC addressing and CRCs.⁴⁵

Consequently, aggressive serialization is mandatory. The architecture implements

key-minification and floating-point precision truncation prior to transmission:

- hex_id maps to the single character h
- elevation_pitch maps to p
- gas_ppm maps to g
- risk_level maps to r

JSON

```
{"h":"88283082b9ffff","p":12.4,"g":405,"r":"High"}
```

This semantic payload compresses massive multidimensional spatial awareness into fewer than 60 bytes.⁴⁶

6.3 Asynchronous Concurrency and I2C Polling

The Python asyncio library isolates the distinct temporal requirements of the hardware layer. Board B dictates DC motor propulsion PWM at 2kHz, while Board A handles servo updates at 50Hz. The MPU6050 demands high-frequency polling to maintain filter accuracy, whereas the Computer Vision loop running OpenCV matrix transformations may incur significant blocking latency. Standard synchronous execution would induce catastrophic timing delays, risking I2C bus lockup and causing mechanical jitter in the propulsion actuators. Asyncio's event loop permits seamless context switching during I/O wait times.⁴⁷

6.4 Python Implementation (main.py)

Python

```
import asyncio
import json
import logging
import board
import busio
import numpy as np
import tensorflow as tf
from adafruit_pca9685 import PCA9685
```

```
logger = logging.getLogger(__name__)
```

```
class RoverController:
```

```
    def __init__(self):
```

```
        """Initializes asynchronous hardware subsystems and Edge AI inference engine."""
```

```

try:
    self.i2c = busio.I2C(board.SCL, board.SDA)

    # Board A: Servos (Steering) @ 50Hz Base Address
    self.pca_servos = PCA9685(self.i2c, address=0x40)
    self.pca_servos.frequency = 50

    # Board B: DC Motors (Torque/Speed) @ 2kHz (Bridged A0 pad)
    self.pca_motors = PCA9685(self.i2c, address=0x41)
    self.pca_motors.frequency = 2000

    logger.info("PCA9685 Dual-Controllers Initialized successfully.")
except Exception as e:
    logger.critical(f"I2C Hardware Initialization Failure: {e}")
    raise

try:
    # Load TFLite Model for Edge Risk Analysis
    self.interpreter = tf.lite.Interpreter(model_path="risk_model.tflite")
    self.interpreter.allocate_tensors()
    self.input_details = self.interpreter.get_input_details()
    self.output_details = self.interpreter.get_output_details()
    logger.info("TensorFlow Lite Edge AI model loaded and tensors allocated.")
except Exception as e:
    logger.warning(f"AI Model load failed, telemetry will default to 'Unknown': {e}")
    self.interpreter = None

def evaluate_risk_tflite(self, ca_grid: np.ndarray) -> str:
    """
    Executes ML inference loop to classify current geographic zone danger level.
    """
    if self.interpreter is None:
        return "Unknown"

    try:
        # Preprocess simulation grid to match model input shape requirement (e.g., 1x50x50x1
float32)
        input_data = ca_grid.astype(np.float32).reshape(1, 50, 50, 1)
        self.interpreter.set_tensor(self.input_details['index'], input_data)

        # Execute hardware inference
        self.interpreter.invoke()
        output_data = self.interpreter.get_tensor(self.output_details['index'])

```

```

        classes = ["Low", "Medium", "High"]
        risk_index = np.argmax(output_data)
        return classes[risk_index]
    except Exception as e:
        logger.error(f"Inference failure during tensor evaluation: {e}")
        return "Error"

def serialize_telemetry(self, hex_id: str, pitch: float, gas: float, risk: str) -> str:
    """
    Compacts multidimensional data into an aggressively minified JSON string
    to respect the physical 256-byte LoRa MTU constraints.
    """
    payload = {
        "h": hex_id,
        "p": round(pitch, 1),
        "g": int(gas),
        "r": risk
    }
    # Dump with no whitespace separation
    json_payload = json.dumps(payload, separators=(',', ':'))

    if len(json_payload.encode('utf-8')) > 250:
        logger.error("Payload violation: String exceeds SX1276 LoRa MTU limit.")
        raise ValueError("Payload size limit exceeded.")

    return json_payload

async def motor_control_loop(self):
    """Asynchronous execution loop for maintaining 2kHz motor translation updates."""
    try:
        while True:
            # Polling kinematics algorithms and applying to pca_motors.channels
            await asyncio.sleep(0.01) # Yield execution back to event loop
    except asyncio.CancelledError:
        logger.info("Motor control loop cancelled.")

async def servo_control_loop(self):
    """Asynchronous execution loop for maintaining 50Hz steering orientation updates."""
    try:
        while True:
            # Polling steering matrix and applying to pca_servos.channels
            await asyncio.sleep(0.02)

```

```

except asyncio.CancelledError:
    logger.info("Servo control loop cancelled.")

async def telemetry_loop(self):
    """Low-frequency asynchronous loop for ML inference and RF transmission."""
    try:
        while True:
            # Simulate CA Grid Output for Demonstration
            dummy_grid = np.random.randint(0, 3, (50, 50))
            risk_level = self.evaluate_risk_tflite(dummy_grid)

            packet = self.serialize_telemetry(
                hex_id="88283082b9ffff",
                pitch=14.2,
                gas=380,
                risk=risk_level
            )

            logger.info(f"Preparing transmission via SX1276 LoRa: {packet}")
            # SPI Write commands to RF96 module would execute here

            await asyncio.sleep(5.0) # Transmit interval
    except asyncio.CancelledError:
        logger.info("Telemetry loop cancelled.")

def cleanup(self):
    """
    Hardware fail-safe: Shuts down all PWM generating signals and releases the I2C bus.
    """
    logger.critical("Initiating absolute hardware shutdown sequence.")
    try:
        for i in range(16):
            self.pca_servos.channels[i].duty_cycle = 0
            self.pca_motors.channels[i].duty_cycle = 0
        self.pca_servos.deinit()
        self.pca_motors.deinit()
        board.I2C().deinit()
        logger.info("I2C Bus de-initialized safely.")
    except Exception as e:
        logger.error(f"Hardware cleanup failed: {e}")

async def main():
    logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -

```

```

%(message)s')
    rover = RoverController()

# Gather and orchestrate concurrent event loops
tasks = asyncio.gather(
    rover.motor_control_loop(),
    rover.servo_control_loop(),
    rover.telemetry_loop()
)

try:
    await tasks
except KeyboardInterrupt:
    logger.warning("System execution interrupted by user.")
    tasks.cancel()
finally:
    rover.cleanup()

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except Exception as e:
        logger.fatal(f"Unrecoverable runtime error: {e}")

```

7. Safety, Lifecycle Management, and Fault Tolerance

Deploying autonomous robotics into remote, unstructured wilderness environments necessitates absolute adherence to fault tolerance methodologies.

1. **I2C Bus Contention and Capacitance Limits:** The system architecture operates multiple discrete I2C boards simultaneously—the ADS1115 (0x48), MPU6050 (0x68), PCA9685 A (0x40), and PCA9685 B (0x41). High cumulative capacitive load on the SCL/SDA signaling lines can cause high-frequency signal degradation, resulting in hardware timeouts and missing ACK bits.⁴⁸ Software architecture mitigates this by wrapping all busio read/writes in rigorous try/except recovery blocks, guaranteeing that an I2C fail state on the IMU does not permanently crash the motor control loop.
2. **Deterministic Shutdown Procedures:** The cleanup() subroutine guarantees that board.deinit() is forcefully executed via Python's finally exception handling mechanism. Failing to explicitly set PCA9685 duty cycles to 0 upon script termination or kernel panic results in "runaway" motors and physical servo lock-up. A runaway 6WD rover draws severe peak current from the LiPo batteries, eventually causing extreme thermal runaway or electrical fires on the L298N sinks.⁴⁹
3. **Impedance Management for Tied Logic Pins:** By physically typing the L298N logic pins

together to conserve GPIO mapping, fewer raw outputs are active. When standard `GPIO.cleanup()` is executed during a script abort, these logic pins transition entirely to high-impedance inputs. This safely ensures that the L298N H-bridge transistors drop to a completely neutral state, disengaging motor torque and allowing the rover chassis to safely coast to a halt rather than violently braking.

8. Conclusion

This blueprint yields a highly cohesive, fault-tolerant mechanical and software control architecture perfectly suited for the computational and hardware constraints inherent to a Raspberry Pi 4. By innovatively tying logic pins together and mathematically compensating through directional vector inversions inside the inverse kinematics engine, the rover successfully conserves crucial IO.

Furthermore, physically segmenting high-frequency 2kHz PWM motor torque control from 50Hz PWM servo angular control onto independent I2C addresses enables completely asynchronous software execution. This non-blocking architecture allows for the unhindered execution of advanced Edge AI and Stochastic Cellular Automata fire modeling. Through the utilization of JSON-compacted, semantic LoRa payload packets and H3 geospatial hexagonal grids, the system guarantees that remote incident monitoring stations receive highly localized, high-value tactical predictive data without saturating the restricted RF spectrum.

Sources des citations

1. L298N Motor driver help - Raspberry Pi Forums, consulté le avril 25, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=68764>
2. L298n motor controller - Raspberry Pi Forums, consulté le avril 25, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=147205>
3. Using Inverse Kinematics to become a Master-Swerver | by Abhinav Peri | Medium, consulté le avril 25, 2026, <https://abhinavwastaken.medium.com/using-inverse-kinematics-to-become-a-master-swerver-1026759d81b0>
4. How to connect dual PXA9685 servo controller to Raspberry pi?, consulté le avril 25, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=351863>
5. Chaining Drivers | Adafruit PCA9685 16-Channel Servo Driver, consulté le avril 25, 2026, <https://learn.adafruit.com/16-channel-pwm-servo-driver/chaining-drivers>
6. Control multiple servos using Raspberry Pi 2 Model B, PCA9685, Windows 10 IOT and C - Mitul Suthar's Coding Blog, consulté le avril 25, 2026, <http://mscodingblog.blogspot.com/2018/01/control-multiple-servos-using-raspberry.html>
7. Connecting the MQ135 gas sensor to the Raspberry Pi 3B+ : r/ECE - Reddit, consulté le avril 25, 2026, https://www.reddit.com/r/ECE/comments/e6guta/connecting_the_mq135_gas_sensor_to_the_raspberry/
8. Wind Vane PCB | Details - Hackaday.io, consulté le avril 25, 2026, <https://hackaday.io/project/5068-low-cost-weather-station/log/22521-wind-vane>

[-pcb](#)

9. SS41F - honeywell - OnlineComponents.com, consulté le avril 25, 2026,
<https://www.onlinecomponents.com/en/productdetail/honeywell/ss41f-11942159.html>
10. SS41F Hall Effect Sensor Datasheet | PDF | Electric Motor | Bipolar Junction Transistor, consulté le avril 25, 2026,
<https://www.scribd.com/document/524554018/4-alan-etkili>
11. Improving MPU6050 IMU Arduino Project Performance with a Complimentary Filter, consulté le avril 25, 2026,
<https://toptechboy.com/improving-mpu6050-imu-arduino-project-performance-with-a-complimentary-filter/>
12. Implementation of Complimentary filter on MPU 6050 - World Scientific News, consulté le avril 25, 2026,
<https://worldscientificnews.com/implementation-of-complimentary-filter-on-mpu-6050/>
13. Vegetation Indices from UAV Imagery: Emerging Tools for Precision Agriculture and Forest Management - MDPI, consulté le avril 25, 2026,
<https://www.mdpi.com/2624-7402/7/12/431>
14. lora-sx1276/README_pySX127x.md at master - GitHub, consulté le avril 25, 2026,
https://github.com/raspberrypi-tw/lora-sx1276/blob/master/README_pySX127x.md
15. pvandervelde/basic-swerve-sim - GitHub, consulté le avril 25, 2026,
<https://github.com/pvandervelde/basic-swerve-sim>
16. SwerveDrive6Kinematics — mostrobotpy documentation, consulté le avril 25, 2026,
<https://robotpy.readthedocs.io/projects/robotpy/en/2024.0.0b3.post2/wpimath.kinematics/SwerveDrive6Kinematics.html>
17. Swerve Drive Kinematics — FIRST Robotics Competition documentation - WPILib, consulté le avril 25, 2026,
<https://docs.wpilib.org/en/stable/docs/software/kinematics-and-odometry/swerve-drive-kinematics.html>
18. Pitch and Roll angles with MPU6050 and Complimentary filter - Sensors - Arduino Forum, consulté le avril 25, 2026,
<https://forum.arduino.cc/t/pitch-and-roll-angles-with-mpu6050-and-complimentary-filter/566844>
19. MPU6050 with arduino complementary filter - Programming, consulté le avril 25, 2026,
<https://forum.arduino.cc/t/mpu6050-with-arduino-complementary-filter/388384>
20. Improving Accuracy of MPU6050 Data Using a Complimentary Filter - Technology Tutorials, consulté le avril 25, 2026,
<https://toptechboy.com/improving-accuracy-of-mpu6050-data-using-a-complimentary-filter/>
21. Complementary filter and relative orientation with MPU6050 - HiBit, consulté le avril 25, 2026,
<https://www.hibit.dev/posts/92/complementary-filter-and-relative-orientation-wit>

[h-mpu6050](#)

22. SS41F|SS41G Series Low-Cost Digital Bipolar Hall-Effect Sensor ICs - Dratek.cz, consulté le avril 25, 2026, <https://dratek.cz/docs/produkty/0/907/41f.pdf>
23. AeroVane: IoT Environmental Station & DIY Wind Direction Sensor (Pico W) - Instructables, consulté le avril 25, 2026, <https://www.instructables.com/AeroVane-IoT-Environmental-Station-DIY-Wind-Direct/>
24. Low-cost Digital Bipolar Hall-effect Sensor ICs,SS41F, SS41G - Mouser Electronics, consulté le avril 25, 2026, <https://www.mouser.com/catalog/specsheets/hwsc-s-a0001295895-1.pdf>
25. Evaluating warm-season turfgrass: Multispectral and red-green-blue index comparison, consulté le avril 25, 2026, <https://gcmonline.com/course/environment/news/evaluating-warm-season-turfgrass-multispectral-and-red-green-blue-index>
26. Modeling global indices for estimating non-photosynthetic vegetation cover, consulté le avril 25, 2026, <https://ntrs.nasa.gov/api/citations/20240008647/downloads/document.pdf>
27. Aggregating and Analyzing Point Data with H3 Hexes and Pandas - Utah Geospatial Resource Center, consulté le avril 25, 2026, <https://gis.utah.gov/blog/2022-10-26-using-h3-hexes/>
28. Uber H3 for Data Analysis with Python | by Eshwaran Venkat | TDS Archive | Medium, consulté le avril 25, 2026, <https://medium.com/data-science/uber-h3-for-data-analysis-with-python-1e54acdcc908>
29. Uber H3 Hexagonal Modeling | HEAVY.AI Documentation, consulté le avril 25, 2026, <https://docs.nvidia.com/heavyai/sql/data-manipulation-dml/geospatial-capabilities/uber-h3-hexagonal-modeling>
30. Statistical dynamics of wildfire burned area from cellular-automata simulators - PMC, consulté le avril 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12783638/>
31. A critical review of wildfire models and simulation tools for WUI applications - NRC Publications Archive, consulté le avril 25, 2026, <https://nrc-publications.canada.ca/eng/view/accepted/?id=8b2eda04-edad-4a94-bcfc-ca1a1792deeb>
32. Using Wind Data to Predict Wildfire Spread in Central California - Saint Mary's University, consulté le avril 25, 2026, <https://gis.smumn.edu/GradProjects/DuF.pdf>
33. Wind and Slope Effects on Wildland Fire Spread: A Review of Experimental, Empirical, Mathematical, and Physics-Based Models - MDPI, consulté le avril 25, 2026, <https://www.mdpi.com/2571-6255/9/3/100>
34. Data-Driven Modeling of Wildfire Spread with Stochastic Cellular Automata and Latent Spatio-Temporal Dynamics - arXiv, consulté le avril 25, 2026, <https://arxiv.org/pdf/2306.03214>
35. An Efficient Method to Simulate Wildfire Propagation Using Irregular Grids - NHESS, consulté le avril 25, 2026,

- <https://nhess.copernicus.org/preprints/nhess-2024-27/nhess-2024-27-ATC1.pdf>
36. Rapid Growth of Large Forest Fires Drives the Exponential Response of Annual Forest-Fire Area to Aridity in the Western United States - PMC, consulté le avril 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9286820/>
 37. I. Introduction II. Methods III. Summary and Conclusions III. Results IV. Acknowledgements and Citations - Department of Atmospheric Sciences, consulté le avril 25, 2026, https://atmos.utah.edu/_resources/documents/pdf_docs/d_lyford_2.pdf
 38. Wind and Slope Influence on Wildland Fire Spread, a Numerical Study - MDPI, consulté le avril 25, 2026, <https://www.mdpi.com/2571-6255/8/6/217>
 39. An Approximated Model of Wildfire Propagation on Slope - arXiv, consulté le avril 25, 2026, <https://arxiv.org/html/2412.18931v1>
 40. Image classification with TensorFlow Lite Model Maker | Google AI Edge, consulté le avril 25, 2026, https://ai.google.dev/edge/litert/libraries/modify/image_classification
 41. joonb14/TFLiteClassification: TensorFlow Lite Image Classification Python Implementation, consulté le avril 25, 2026, <https://github.com/joonb14/TFLiteClassification>
 42. Running TensorFlow Lite Image Classification Models in Python - Comet, consulté le avril 25, 2026, <https://www.comet.com/site/blog/running-tensorflow-lite-image-classification-models-in-python/>
 43. Exploring LoRa RYLR896 with MicroPython - YouTube, consulté le avril 25, 2026, https://www.youtube.com/watch?v=RAvnOMW_P-s
 44. How to send thousands of data points in a single LoRaWAN packet? - Stack Overflow, consulté le avril 25, 2026, <https://stackoverflow.com/questions/68943507/how-to-send-thousands-of-data-points-in-a-single-lorawan-packet>
 45. Compression strategies for LoRaWAN packets - The Things Network, consulté le avril 25, 2026, <https://www.thethingsnetwork.org/forum/t/compression-strategies-for-lorawan-packets/42452>
 46. chandrawi/LoRaRF-Python: Python library for basic transmitting and receiving data using LoRa and FSK modem - GitHub, consulté le avril 25, 2026, <https://github.com/chandrawi/LoRaRF-Python>
 47. pca 9685 not working with raspberry pi 4, consulté le avril 25, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=391924>
 48. Using multiple i2c port - Raspberry Pi Forums, consulté le avril 25, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=349089>
 49. Servo Driver PCA9685 With Raspberry Pi | Multiple Servos Library Installation - YouTube, consulté le avril 25, 2026, <https://www.youtube.com/watch?v=-x2EEIUMm6A>